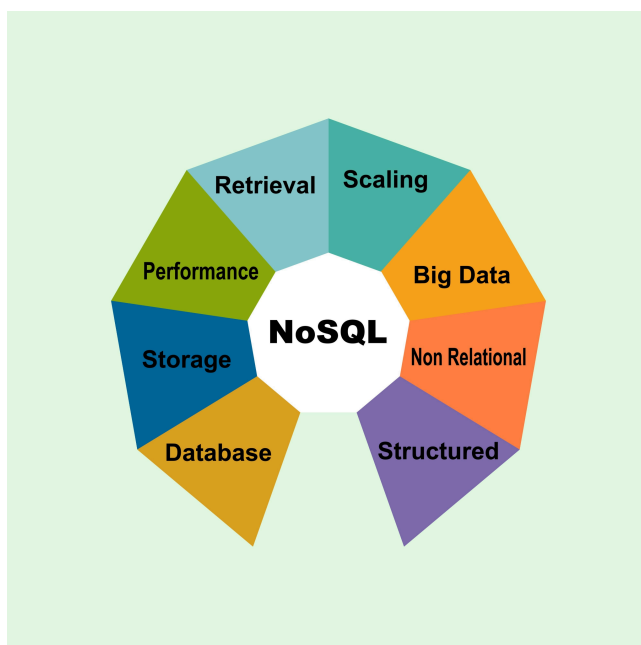


Arquitectura general de bases de datos no relacionales

Introducción

Las bases de datos no relacionales, o NoSQL, surgieron como una alternativa a las bases de datos relacionales tradicionales (SQL) para manejar grandes volúmenes de datos no estructurados o semiestructurados. Su arquitectura está diseñada para ser flexible, distribuida y escalable.

A diferencia de las bases de datos SQL que utilizan un esquema rígido de tablas, filas y columnas, las bases de datos NoSQL emplean diversos modelos de datos.



Las características clave de la arquitectura NoSQL son:

Esquemas Flexibles: Los documentos en una misma colección (el equivalente a una tabla en SQL) pueden tener estructuras diferentes. Esto permite una evolución más rápida de las aplicaciones, ya que no es necesario alterar un esquema predefinido para almacenar nuevos tipos de datos.

Distribución de Datos: Las bases de datos NoSQL están diseñadas para ejecutarse en clústeres de servidores, distribuyendo los datos entre múltiples nodos. Esto es fundamental para la escalabilidad y la alta disponibilidad.

Sin JOINS Complejos: Generalmente, las bases de datos NoSQL evitan las operaciones de JOIN complejas que son comunes en SQL. En su lugar, se fomenta la desnormalización de los datos, donde la información relacionada se anida dentro de un mismo documento para agilizar las consultas.

Replicación: Alta Disponibilidad y Tolerancia a Fallos

La replicación es el proceso de crear y mantener múltiples copias de los datos en diferentes servidores o nodos. Es una característica fundamental en las bases de datos NoSQL para garantizar la alta disponibilidad y la tolerancia a fallos. Si un servidor falla, las otras copias (réplicas) siguen estando disponibles para atender las solicitudes.

En MongoDB, la replicación se implementa a través de conjuntos de réplicas (replica sets). Un conjunto de réplicas es un grupo de servidores que mantienen el mismo conjunto de datos. Dentro de un conjunto de réplicas, existen dos roles principales:

Primario (Master): Es el único nodo que recibe todas las operaciones de escritura. Es el punto de referencia principal para los datos.

Secundarios (Slaves): Replican el conjunto de datos del primario. Las operaciones de lectura se pueden distribuir entre los nodos secundarios para balancear la carga.

Este modelo se conoce como replicación maestro-esclavo. Si el nodo primario falla, los nodos secundarios inician un proceso de "elección" para designar a un nuevo primario de entre ellos, asegurando que la base de datos siga operativa para las escrituras.

Existen dos modos principales de replicación:

Replicación Síncrona: La operación de escritura se considera completa solo después de que se haya confirmado en el primario y en todas las réplicas. Garantiza la máxima consistencia, pero puede introducir latencia.

Replicación Asíncrona: La operación de escritura se considera completa una vez que se ha confirmado en el primario, y luego se propaga a las réplicas en segundo plano. Ofrece un mejor rendimiento, pero existe una pequeña ventana de tiempo en la que las réplicas pueden no tener los datos más recientes. MongoDB, por defecto, utiliza la replicación asíncrona.

Escalabilidad: Creciendo con la Demanda

Una de las ventajas más significativas de las bases de datos NoSQL es su capacidad para escalar y manejar cargas de trabajo crecientes. Existen dos formas principales de escalar un sistema de base de datos:

Escalabilidad Vertical (Scale-Up)

La escalabilidad vertical consiste en aumentar los recursos de un único servidor, como añadir más CPU, memoria RAM o un almacenamiento más rápido. Es como cambiar el motor de un coche por uno más potente.

Ventajas: Es simple de implementar.

Desventajas: Tiene un límite físico (no se puede añadir hardware indefinidamente) y puede llegar a ser muy costoso. Además, implica un único punto de fallo.

Escalabilidad Horizontal (Scale-Out)

La escalabilidad horizontal implica añadir más servidores a un clúster para distribuir la carga de trabajo entre ellos. En lugar de un coche más potente, se añaden más coches a la flota. Esta es la estrategia preferida por las bases de datos NoSQL como MongoDB.

Ventajas: Ofrece una escalabilidad casi ilimitada a un costo menor, utilizando hardware estándar. Mejora la tolerancia a fallos, ya que la caída de un nodo no afecta al sistema en su conjunto.

Desventajas: Aumenta la complejidad de la gestión y el mantenimiento.

En MongoDB, la escalabilidad horizontal se logra a través del sharding (fragmentación). El sharding consiste en dividir grandes conjuntos de datos en fragmentos más pequeños y distribuirlos entre múltiples servidores (o "shards"). Cada shard contiene una porción de los datos totales. Cuando una aplicación realiza una consulta, un proceso de enrutamiento (mongos) dirige la consulta al shard o shards correctos.

Limitaciones y Consideraciones

A pesar de sus ventajas, las bases de datos NoSQL tienen limitaciones que es importante conocer:

Consistencia de Datos: A diferencia de las bases de datos SQL que garantizan la consistencia ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad), muchas bases de datos NoSQL siguen el principio BASE (Basically Available, Soft state, Eventually consistent). Esto significa que priorizan la disponibilidad sobre la consistencia inmediata. Los datos se volverán consistentes en todas las réplicas "eventualmente", pero puede haber breves períodos de inconsistencia. Esto puede no ser adecuado para todas las aplicaciones, como las transacciones financieras.

Transacciones Complejas: El soporte para transacciones que involucran múltiples documentos o colecciones es limitado o inexistente en muchas bases de datos NoSQL. Aunque MongoDB ha mejorado significativamente su soporte para transacciones multi-documento en versiones recientes, no es su punto fuerte principal en comparación con los sistemas SQL diseñados para ello.

Falta de un Lenguaje de Consulta Estándar: Mientras que SQL es el estándar de facto para las bases de datos relacionales, cada base de datos NoSQL tiene su propio lenguaje y API de consulta. Esto puede implicar una curva de aprendizaje si se cambia de una tecnología NoSQL a otra.

Complejidad en las Consultas: Las consultas que requieren JOINS complejos entre diferentes conjuntos de datos pueden ser ineficientes o difíciles de implementar en

NoSQL. El diseño del esquema de datos (la desnormalización) es crucial para optimizar las consultas.